



1. 算法的空间复杂度表示的是

- A. 算法的可读性
- B. 算法的难易程度
- C. 执行算法所耗费的时间
- D. 执行算法所耗费的存储空间

2. 对需要频繁插入和删除元素的线性表，适合的存储方式是

- A. 顺序存储
- B. 链式存储
- C. 索引存储
- D. 散列存储

3. 线性表的两个元素，如果逻辑上相邻，则

- A. 顺序存储和链式存储时都一定相邻
- B. 顺序存储和链式存储时都一定不相邻
- C. 顺序存储时一定相邻，链式存储时不一定相邻
- D. 顺序存储时不一定相邻，链式存储时一定相邻

4. 在头指针为head的单链表中，判断指针变量p指向终端结点的条件是

- A. $p \rightarrow next \rightarrow next == head$
- B. $p \rightarrow next == head$
- C. $p \rightarrow next \rightarrow next == NULL$
- D. $p \rightarrow next == NULL$

5. 若栈的进栈序列为5,4,3,2,1，则经过出入栈操作可能获得的出栈序列是

- A. 4,5,1,3,2
- B. 3,5,4,2,1
- C. 2,1,3,5,4
- D. 4,3,5,1,2

6. 在三维数组a[7][4][10]中，每个数组元素占用2个存储单元，所有数组元素存放在一个连续的存储空间中，则该数组需要的存储单元总数是

- A. 560
- B. 280
- C. 42
- D. 21

7. 下列广义表中，表长为3的是

- A. ((a,b,c))
- B. (a,(b,c),(d,e,f))
- C. (a,b,c,(d,e,f))
- D. (a,(b,c,d,e,f))

8. 深度为k($k \geq 1$)的满二叉树所包含的结点数是

- A. $k+1$

- B. 2^k
- C. 2^{k-1}
- D. 2^{k+1}

9. 下列选项中，能唯一确定一棵二叉树的两个遍历序列是

- A. 前序遍历序列和层次遍历序列
- B. 后序遍历序列和层次遍历序列
- C. 前序遍历序列和中序遍历序列
- D. 前序遍历序列和后序遍历序列

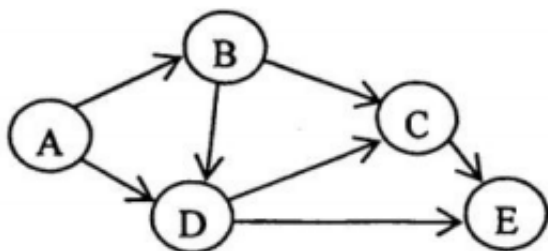
10. 下列关于连通的无向带权图G的叙述中，正确的是

- A. 图G的生成树至少含有一个回路
- B. 图G的最小生成树总是唯一的
- C. 图G的邻接矩阵不一定是对称矩阵
- D. 图G的生成树的边数等于顶点数减1

11. 下列排序方法中，不稳定的是

- A. 堆排序
- B. 冒泡排序
- C. 归并排序
- D. 直接插入排序

12. 对图进行拓扑排序，可以得到的拓扑序列是



- A. BADCEB
- B. BACDEA
- C. ACBDED>E
- D. ABDCE

13. 下列选项中，能使用二分查找算法的是

- A. 顺序存储的线性表(4,16,5,6,55,89,34,25)
- B. 顺序存储的线性表(4,5,6,16,25,34,55,89)
- C. 散列存储的线性表(4,5,6,16,25,34,55,89)
- D. 链式存储的线性表(4,5,6,16,25,34,55,89)

14. 在下列查找算法中，平均查找长度与数据规模基本无关的是

- A. 顺序查找
- B. 散列查找
- C. 二分查找
- D. B树中的查找

15. 假设散列表长 $m=10$, 散列函数 $H(\text{key})=\text{key}\%9$ 。表中已有3个结点： $H(23)=5, H(31)=4, H(17)=8$, 其余位置为空。现采用线性探查法处理冲突，依次存储关键字4和36时需要探查的次数分别是A.1和1

02331 数据结构 2023年4月

- B.2和1
- C.3和1
- D.1和3

16.顺序存储和链接存储方法中，无需连续分配存储空间的是（ ）。

17.设顺序表首元素的存储地址是4000，每个数据元素占8个存储单元，则第11个元素的存储地址是（ ）。

18.若在长度为n的顺序表中删除第i个元素($1 \leq i \leq n$),则需要向前移动的元素个数是（ ）。

19.顺序栈存放在数组S[m]中，S[m-1]保存栈底元素，用栈顶指针top==m表示栈空，则栈满的条件是（ ）。

20.限制在表的一端插入数据、在表的另一端删除数据的线性表是（ ）。

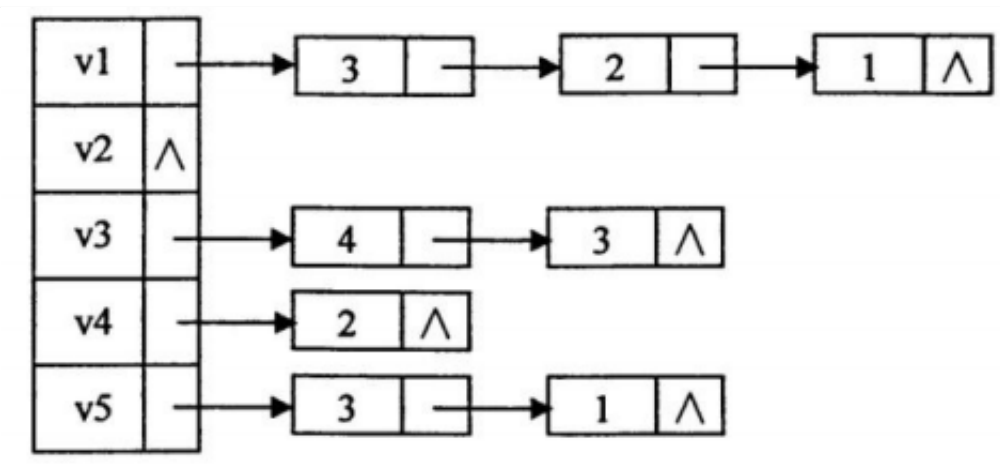
21.广义表A=(a,b,c,(e,f,g,h),head(tail(tail(A)))=（ ）。

22.以权值分别为1,3,5,7的四个叶子结点构成的哈夫曼树，其带权路径长度WPL是（ ）。

23.若选用的排序算法不稳定，则关键字相同的两个记录在排序前后的相对次序（ ）。

24.由m个结点构成的二叉排序树，其可能的最大深度是（ ）。

25.已知图G的邻接表A如题图所示。



根据A中边表的次序，从顶点v3出发进行深度优先搜索遍历，得到的深度优先搜索序列是（ ）。

26.设循环队列保存在数组Q[N]中，front和rear分别为队头和队尾指针，初始时front=rear=0,约定指针rear指向的单元始终为空，请用C语言分别描述下列操作：(1)将数据元素x入队。(2)将队首元素出队，并保存到变量myData中。(3)计算队列中当前数据元素的个数，并保存到变量DLenth中。

27.已知稀疏矩阵M如下，采用三元组表存储。

$$M_{7 \times 5} = \begin{bmatrix} 0 & 0 & 22 & 0 & -11 \\ 33 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -22 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 88 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 77 \end{bmatrix}$$

- (1)请给出三元组表的类型定义。
- (2)写出矩阵M按列优先存储的三元组表。

28.已知待排序记录的关键字序列为(45,37,75,18,53,31,48,37)，请回答下列问题。

- (1)画出其对应的完全二叉树T。
- (2)将T调整为对应的大根堆，给出大根堆序列。

29.将百分制成绩分成五个等级，已知成绩的对应关系及分布情况如下表所示。

百分制成绩 s	0~59	60~74	75~84	85~94	95~100
等级 grade	E	D	C	B	A
百分比	5	25	45	20	5

请回答下列问题。
(1)根据哈夫曼树的基本原理，画出成绩评定判定树T。
(2)求树T的WPL。

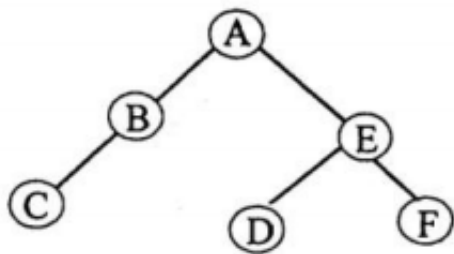
30.单链表类型定义如下：

```
typedef struct node{
DataType data;
struct node *next;
}LinkNode;
typedef LinkNode *Linklist;
函数f30的功能是删除带头结点的单链表中data值为x的全部结点，请在空白处填上适当内容将算法补充完整。
void f30(Linklist head,DataTypex)
{
LinkNode *p,*q,*s;
p=head;q= (1) ;
while(q!=NULL)
if( (2) ){
s=q;q=q->next;
p->next=q;free(s);
}
else{
p=q;
q= (3) ;
}
}
```

31.二叉树的二叉链表类型定义如下：

```
#define char DataType
typedef struct node{
DataType data;
struct node *lchild,*rchild;
}BinTNode;
typedef BinTNode *BinTree;
阅读下列函数并回答问题。
void f31(BinTree bt){
if(bt!=NULL){
f31(bt->rchild);
f31(bt->lchild);
printf("%c",bt->data);
}
}
```

(1)给出如图所示的二叉树T,写出执行函数f31(T)后得到的输出序列。
(2)对于二叉树中的任意结点N及它的左子树L和它的右子树R,f31的遍历次序是什么？



32.阅读下列函数并回答问题。

```
void f32(int r[],int N){
```

```
int i,j,temp;
```

```
for(i=1;i<N;i++){
```

```
    (3)q->next
```

31.

答：(1)FDECBA

(2)RLN次序(或先序遍历的逆)

32.

答：(1)3,4,5,6,9,12,35,78

(2)直接插入排序

33.

答：(1) $R[mid].key > k$

(2)-1和5

34.

答：`#define Min -65525`

```
int f34(BinTree BT)
```

```
{
```

```
int lvalue,rvalue,maxvalue;
```

```
if(BT==NULL)return Min;
```

```
if(BT!=NULL)
```

```
{
```

```
lvalue=f34(BT->lchild);
```

```
rvalue=f34(BT->rchild);
```

```
maxvalue=(lvalue>rvalue)?lvalue:rvalue;
```

```
maxvalue=(maxvalue>BT->data)?maxvalue:BT->data;
```

```
return maxvalue;
```

```
}
```